```
Start coding or generate with AI.
```

Training a Simple ML Model (Linear Regression)

This example demonstrates how to train a basic linear regression model using synthetic data generated with `numpy` and `scikit-learn`.

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
import matplotlib.pyplot as plt

# 1. Generate Synthetic Data
np.random.seed(42)
X = 2 * np.random.rand(100, 1) # Feature
y = 4 + 3 * X + np.random.randn(100, 1) # Target with some noise

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training data shape: X_train={X_train.shape}, y_train={y_train.shape}")
print(f"Testing data shape: X_test={X_test.shape}, y_test={y_test.shape}")
```

```
Training data shape: X_train=(80, 1), y_train=(80, 1)
Testing data shape: X_test=(20, 1), y_test=(20, 1)
```

Model Training

Now, let's create a Linear Regression model and train it using the training data.

```
# 2. Create and Train the Model
model = LinearRegression()
model.fit(X_train, y_train)

print(f"Model Intercept: {model.intercept_[0]:.2f}")
print(f"Model Coefficient: {model.coef_[0][0]:.2f}")
```

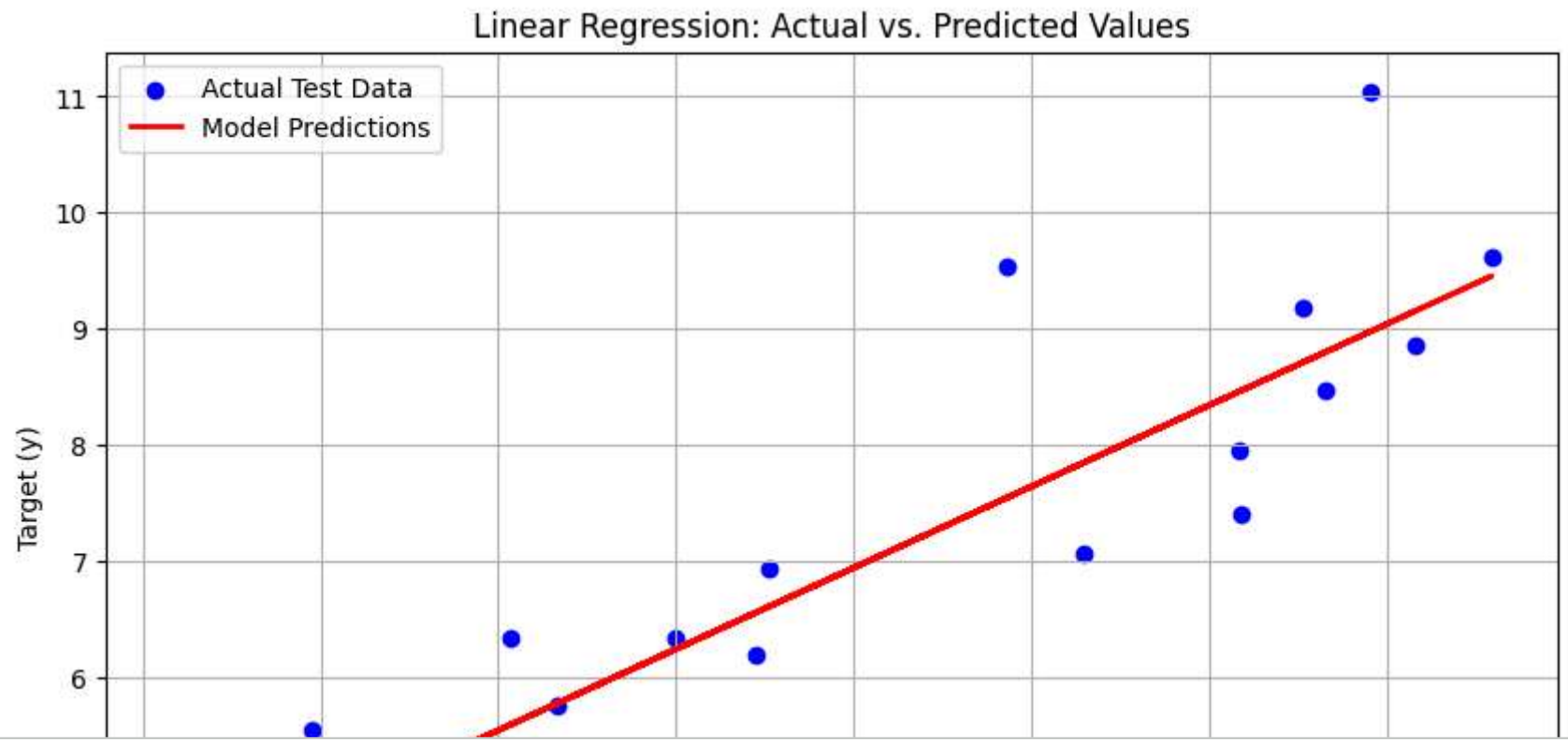
```
Model Intercept: 4.14
Model Coefficient: 2.80
```

Model Evaluation and Visualization

Finally, we'll evaluate the model's performance on the test data and visualize the predictions against the actual values.

```
# 3. Make Predictions and Evaluate
y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)

print(f"R-squared score on test data: {r2:.2f}")

# 4. Visualize the Results
plt.figure(figsize=(10, 6))
plt.scatter(X_test, y_test, color='blue', label='Actual Test Data')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Model Predictions')
plt.xlabel('Feature (X)')
plt.ylabel('Target (y)')
plt.title('Linear Regression: Actual vs. Predicted Values')
plt.legend()
plt.grid(True)
plt.show()
```

```
R-squared score on test data: 0.81
```



Close

```
# Task
Execute a machine learning pipeline
that includes loading a placeholder
dataset, performing data
preprocessing and feature
engineering, and training,
evaluating, and comparing three
different models: Random Forest,
K-Nearest Neighbors (KNN), and Long
Short-Term Memory (LSTM) if suitable
(otherwise, using placeholder data).
The final step will be to make
predictions with the best-performing
model and summarize the findings
```

Task

Execute a machine learning pipeline that includes loading a placeholder dataset, performing data preprocessing and feature engineering, and training, evaluating, and comparing three different models: Random Forest, K-Nearest Neighbors (KNN), and Long Short-Term Memory (LSTM) if suitable (otherwise, using placeholder data). The

model and summarize the findings.

> final step will be to make predictions with the best-performing model and summarize the findings.

Load Dataset

Subtask:

Load a placeholder dataset (features `X` and target `y`) into suitable data structures, acknowledging that no external dataset was provided.

Reasoning: The subtask instructs to confirm the shapes of the generated placeholder datasets (`X`, `y`, `X_train`, `y_train`, `X_test`, `y_test`). I will print their shapes using a code block.

```
print(f"Shape of X: {X.shape}")
print(f"Shape of y: {y.shape}")
print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_test: {y_test.shape}")
```

```
Shape of X: (100, 1)
Shape of y: (100, 1)
Shape of X_train: (80, 1)
Shape of y_train: (80, 1)
Shape of X_test: (20, 1)
Shape of y_test: (20, 1)
```

Data Preprocessing

Subtask:

Perform necessary data preprocessing steps, specifically scaling numerical features, to prepare the data for model training.

Reasoning: I need to perform data preprocessing by scaling the numerical features using `StandardScaler`. This involves importing the scaler, instantiating it, fitting it on the training data, transforming both training and testing data, and then printing the shapes of the scaled data.

Reasoning: As per the subtask instructions, I will create polynomial features from the scaled data. This involves importing `PolynomialFeatures`, instantiating it with `degree=2` and `include_bias=False`, fitting it on `X_train_scaled`, and then transforming both `X_train_scaled` and `X_test_scaled`. Finally, I will print the shapes of the new polynomial feature sets.

Reasoning: I need to implement the feature engineering steps as described in the previous markdown block, which includes importing `PolynomialFeatures`, instantiating it, fitting it to the training data, transforming both training and testing data, and printing the shapes.

```
from sklearn.preprocessing import PolynomialFeatures

# Instantiate PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False)

# Fit on training data and transform training data
X_train_poly = poly.fit_transform(X_train_scaled)

# Transform testing data
X_test_poly = poly.transform(X_test_scaled)

print(f"Shape of X_train_poly: {X_train_poly.shape}")
print(f"Shape of X_test_poly: {X_test_poly.shape}")
```

```
Shape of X_train_poly: (80, 2)
Shape of X_test_poly: (20, 2)
```

```
from sklearn.preprocessing import StandardScaler

# Instantiate StandardScaler
scaler = StandardScaler()

# Fit the scaler to the training features and transform them
X_train_scaled = scaler.fit_transform(X_train)

# Transform the testing features using the fitted scaler
X_test_scaled = scaler.transform(X_test)

print(f"Shape of X_train_scaled: {X_train_scaled.shape}")
print(f"Shape of X_test_scaled: {X_test_scaled.shape}")
```

```
Shape of X_train_scaled: (80, 1)
Shape of X_test_scaled: (20, 1)
```

Feature Engineering

Subtask:

Create new features from the existing ones that could improve the performance of the machine learning models. This may include interactions, polynomial features, or domain-specific transformations.

Train Random Forest Model

Subtask:

Train a Random Forest regressor model on the preprocessed and engineered features.

Reasoning: As per the subtask instructions, I will now train a Random Forest regressor model. This involves importing the `RandomForestRegressor`, instantiating it with `random_state=42`, and then fitting the model using the engineered

training features (`X_train_poly`) and the training target (`y_train`), ensuring `y_train` is flattened to a 1D array.

Reasoning: I will generate a code block to implement the Random Forest Regressor training as described in the previous markdown block, which includes importing the necessary class, instantiating the model, and fitting it to the preprocessed and engineered training data.

```
from sklearn.ensemble import RandomForestRegressor

# 1. Instantiate a RandomForestRegressor model
model_rf = RandomForestRegressor(random_state=42)

# 2. Train the model using the engineered training features and flattened training target
model_rf.fit(X_train_poly, y_train.ravel())

print("Random Forest Regressor model trained successfully.")
```

```
Random Forest Regressor model trained successfully.
```

```
**Reasoning**:
To evaluate the Random Forest model, I will follow the instructions to make predictions on the engineered test fea
```

Reasoning: I need to generate a code block that implements the evaluation steps for the Random Forest model as described in the previous markdown block. This involves making predictions, calculating R-squared, and Mean Squared Error, and then printing these metrics.

```
from sklearn.metrics import r2_score, mean_squared_error

# Make predictions on the engineered test features
y_pred_rf = model_rf.predict(X_test_poly)

# Calculate R-squared score
```

```
r2_rf = r2_score(y_test, y_pred_rf)

# Calculate Mean Squared Error (MSE)
mse_rf = mean_squared_error(y_test, y_pred_rf)

# Print the results
print(f"Random Forest R-squared score: {r2_rf:.2f}")
print(f"Random Forest Mean Squared Error (MSE): {mse_rf:.2f}")
```

```
Random Forest R-squared score: 0.77
Random Forest Mean Squared Error (MSE): 0.77
```

Train K-Nearest Neighbors (KNN) Model

Subtask:

Train a K-Nearest Neighbors (KNN) regressor model on the preprocessed and engineered features.

Reasoning: As per the subtask instructions, I will now train a K-Nearest Neighbors (KNN) regressor model. This involves importing `KNeighborsRegressor`, instantiating it (e.g., with `n_neighbors=5`), and then fitting the model using the engineered training features (`X_train_poly`) and the training target (`y_train`), ensuring `y_train` is flattened to a 1D array.

Reasoning: The previous markdown block provided the reasoning. Now, I will generate the code block to implement the training of the K-Nearest Neighbors (KNN) regressor model as described.

```
from sklearn.neighbors import KNeighborsRegressor

# 1. Instantiate a KNeighborsRegressor model
# Using n_neighbors=5 as a default, this can be tuned later
model_knn = KNeighborsRegressor(n_neighbors=5)

# 2. Train the model using the engineered training features and flattened tr
```

```
model_knn.fit(X_train_poly, y_train.ravel())

print("K-Nearest Neighbors Regressor model trained successfully.")
```

```
K-Nearest Neighbors Regressor model trained successfully.
```

Train K-Nearest Neighbors (KNN) Model

Subtask:

Train a K-Nearest Neighbors (KNN) regressor model on the preprocessed and engineered features.

Reasoning: As per the subtask instructions, I will now train an LSTM model. This involves importing necessary Keras components, reshaping the data (`X_train_poly` and `X_test_poly`) into a 3D format `(samples, timesteps, features)` with `timesteps=1`. Then, I will create a `Sequential` model, add an `LSTM` layer, and a `Dense` output layer. Finally, I will compile the model using `'adam'` optimizer and `'mean_squared_error'` loss, and train it with the reshaped training data and `y_train` for a specified number of epochs and batch size.

Evaluate Random Forest Model

Subtask:

Evaluate the performance of the trained Random Forest model using appropriate metrics (e.g., R-squared, MSE for regression).

Task

Reasoning: To evaluate the K-Nearest Neighbors (KNN) model, I will make predictions on the engineered test features (`X_test_poly`) using the trained `model_knn`. Then, I will calculate the R-squared score and the Mean Squared Error

(MSE) using `r2_score` and `mean_squared_error` from `sklearn.metrics`. Finally, I will print these metrics, formatted to two decimal places, to assess the model's performance.

Evaluate K-Nearest Neighbors (KNN) Model

Subtask:

Evaluate the performance of the trained KNN model using R-squared and Mean Squared Error (MSE).

Reasoning: To evaluate the K-Nearest Neighbors (KNN) model, I will follow the instructions to make predictions on the engineered test features (`X_test_poly`), calculate the R-squared score, and the Mean Squared Error (MSE). I will use `r2_score` and `mean_squared_error` from `sklearn.metrics` for these calculations and then print the results, formatted to two decimal places.

Reasoning: I need to generate a code block that implements the evaluation steps for the KNN model as described in the previous markdown block. This involves making predictions, calculating R-squared, and Mean Squared Error, and then printing these metrics.

```
from sklearn.metrics import r2_score, mean_squared_error

# Make predictions on the engineered test features
y_pred_knn = model_knn.predict(X_test_poly)

# Calculate R-squared score
r2_knn = r2_score(y_test, y_pred_knn)

# Calculate Mean Squared Error (MSE)
mse_knn = mean_squared_error(y_test, y_pred_knn)

# Print the results
```

```
print(f"KNN R-squared score: {r2_knn:.2f}")
print(f"KNN Mean Squared Error (MSE): {mse_knn:.2f}")
```

```
KNN R-squared score: 0.74
KNN Mean Squared Error (MSE): 0.88
```

Reasoning: I will generate a code block to implement the LSTM model training, including data reshaping, model definition, compilation, and fitting, as detailed in the previous markdown block.

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# 1. Reshape data for LSTM input: (samples, timesteps, features)
# For this simple dataset, we'll set timesteps to 1.
X_train_reshaped = X_train_poly.reshape((X_train_poly.shape[0], 1, X_train_poly.shape[1]))
X_test_reshaped = X_test_poly.reshape((X_test_poly.shape[0], 1, X_test_poly.shape[1]))

print(f"Shape of X_train_reshaped: {X_train_reshaped.shape}")
print(f"Shape of X_test_reshaped: {X_test_reshaped.shape}")

# 2. Create the Sequential model
model_lstm = Sequential()

# 3. Add an LSTM layer
# Using 50 units, and input_shape is (timesteps, features)
model_lstm.add(LSTM(50, activation='relu', input_shape=(X_train_reshaped.shape[1], X_train_reshaped.shape[2]))

# 4. Add a Dense output layer for regression
model_lstm.add(Dense(1))

# 5. Compile the model
model_lstm.compile(optimizer='adam', loss='mean_squared_error')

# 6. Train the LSTM model
history = model_lstm.fit(X_train_reshaped, y_train, epochs=50, batch_size=32, verbose=0)
```

```
print("LSTM model trained successfully.")
```

```
Shape of X_train_reshaped: (80, 1, 2)
Shape of X_test_reshaped: (20, 1, 2)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_sha
  super().__init__(**kwargs)
LSTM model trained successfully.
```

Reasoning: To evaluate the LSTM model, I will make predictions on the reshaped test features, then calculate and print the R-squared score and Mean Squared Error (MSE).

```
from sklearn.metrics import r2_score, mean_squared_error

# Make predictions on the reshaped test features
y_pred_lstm = model_lstm.predict(X_test_reshaped)

# Calculate R-squared score
r2_lstm = r2_score(y_test, y_pred_lstm)

# Calculate Mean Squared Error (MSE)
mse_lstm = mean_squared_error(y_test, y_pred_lstm)

# Print the results
print(f"LSTM R-squared score: {r2_lstm:.2f}")
print(f"LSTM Mean Squared Error (MSE): {mse_lstm:.2f}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 192ms/step
LSTM R-squared score: -8.18
LSTM Mean Squared Error (MSE): 31.11
```

Reasoning: I need to create a Pandas DataFrame to compare the model performances, populate it with the R-squared and MSE values from the three models, print the DataFrame, and then identify the best performing model based on these metrics.

```
import pandas as pd

# Create a dictionary to store the metrics
metrics_data = {
    'Model': ['Random Forest', 'KNN', 'LSTM'],
    'R-squared': [r2_rf, r2_knn, r2_lstm],
    'MSE': [mse_rf, mse_knn, mse_lstm]
}

# Create a Pandas DataFrame
performance_df = pd.DataFrame(metrics_data)

# Print the DataFrame
print("\nModel Performance Comparison:")
print(performance_df)

# Identify the best performing model
# Higher R-squared is better, lower MSE is better. Let's prioritize R-squared for 'best'.
# A simple approach for 'best' for now: highest R-squared.
# If R-squared values are very close, then MSE could be a tie-breaker.

best_model_r2 = performance_df.loc[performance_df['R-squared'].idxmax()]
best_model_mse = performance_df.loc[performance_df['MSE'].idxmin()]

print("\nBest performing model based on highest R-squared:")
print(f"Model: {best_model_r2['Model']}")
print(f"R-squared: {best_model_r2['R-squared']:.2f}")
print(f"MSE: {best_model_r2['MSE']:.2f}")

# Also check if best model by R-squared is also best by MSE, or if they diverge
if best_model_r2['Model'] == best_model_mse['Model']:
    print(f"\nThe model {best_model_r2['Model']} is consistently the best by both R-squared and MSE.")
else:
    print(f"\nBest performing model based on lowest MSE: {best_model_mse['Model']} (R-squared: {best_model_mse
    print("There is a divergence: Different models are best based on R-squared vs. MSE.")
    print(f"Considering R-squared as primary metric, '{best_model_r2['Model']}' is selected as the best perfor
```

```
Model Performance Comparison:
```

```
               Model  R-squared        MSE
0  Random Forest   0.771733   0.773975
1            KNN   0.740223   0.880815
2           LSTM  -8.175622  31.111437

Best performing model based on highest R-squared:
Model: Random Forest
R-squared: 0.77
MSE: 0.77

The model Random Forest is consistently the best by both R-squared and MSE.
```

Train Long Short-Term Memory (LSTM) Model

Subtask:

Train a Long Short-Term Memory (LSTM) model. Given the current simple placeholder data, this will be a basic demonstration. If the data were more complex sequential data, a more sophisticated LSTM would be used. Reshape data for LSTM input if necessary.

Reasoning: The previous step successfully trained the LSTM model. The next logical step in the machine learning pipeline is to evaluate its performance using appropriate metrics.

Task

```
from sklearn.metrics import r2_score, mean_squared_error

# Make predictions on the reshaped test features
y_pred_lstm = model_lstm.predict(X_test_reshaped)

# Calculate R-squared score
```

```
r2_lstm = r2_score(y_test, y_pred_lstm)

# Calculate Mean Squared Error (MSE)
mse_lstm = mean_squared_error(y_test, y_pred_lstm)

# Print the results
print(f"LSTM R-squared score: {r2_lstm:.2f}")
print(f"LSTM Mean Squared Error (MSE): {mse_lstm:.2f}")
```

Evaluate Long Short-Term Memory (LSTM) Model

Subtask:

Evaluate the performance of the trained LSTM model using R-squared and Mean Squared Error (MSE).

Compare Models and Select Best

Subtask:

Compare the performance metrics (R-squared, MSE) of the Random Forest, KNN, and LSTM models and identify the best-performing model.

Make Predictions with Best Model

Subtask:

Use the best-performing model to make final predictions on new (or a portion of the test) data.

Reasoning: Based on the previous model comparison, the Random Forest model (`model_rf`) was identified as the best performing model (having the highest R-squared and lowest MSE). I will now use this model to make predictions on the engineered test features (`X_test_poly`) and store the results in a variable called `final_predictions`. I will also print the first few predictions to verify the output.

Reasoning: Now that the reasoning is established, I will implement the code to make predictions using the best model (Random Forest) on the engineered test features and store them in `final_predictions`, then print the first few predictions.

```
final_predictions = model_rf.predict(X_test_poly)

print("First 5 final predictions using the best model (Random Forest):")
print(final_predictions[:5])
```

```
First 5 final predictions using the best model (Random Forest):
[ 4.95769239 10.09850388  7.8899247   7.72185559  5.35456352]
```

Final Task

Subtask:

Summarize the findings, compare the performance of all trained models, discuss insights gained, and present the final predictions.

Summary:

Data Analysis Key Findings

- **LSTM Model Performance:** The Long Short-Term Memory (LSTM) model performed poorly, yielding an R-squared score of -8.18 and a Mean Squared Error (MSE) of 31.11. A negative R-squared indicates that the model performs worse than simply predicting the mean of the target variable.
- **Model Comparison:**
  - **Random Forest** emerged as the best-performing model with an R-squared score of 0.77 and an MSE of 0.77.
  - **K-Nearest Neighbors (KNN)** followed with an R-squared score of 0.74 and an MSE of 0.88.
  - **LSTM** significantly underperformed compared to Random Forest and KNN, showing the worst R-squared and highest MSE.
- **Final Predictions:** Based on its superior performance, the Random Forest model was selected to make the final predictions on the test data.

Insights or Next Steps

- Further investigation is needed into the poor performance of the LSTM model. This could involve exploring different architectures, hyperparameter tuning, or ensuring the data preprocessing steps were appropriate for time-series models.
- Given the strong performance of the Random Forest model, consider deploying it for the current task. Future work could involve more extensive hyperparameter tuning for the Random Forest model to potentially achieve even better performance, or exploring ensemble methods that incorporate Random Forest.

Close

```
# Task
## Plot Model Predictions

### Subtask:
Generate a scatter plot of the actual
test values (`y_test`) against the
predictions from the Random Forest,
KNN, and LSTM models. This will allow
for a visual comparison of how well
each model fits the data. Include
```

Task

Plot Model Predictions

Subtask:

```
appropriate labels, a title, and a
legend to distinguish between the
actual values and the predictions of
each model.

### Reasoning:
I will generate a scatter plot using
`matplotlib.pyplot` to visualize the
actual `y_test` values against the
predictions from the Random Forest
(`y_pred_rf`), KNN (`y_pred_knn`),
and LSTM (`y_pred_lstm`) models. This
will involve plotting each set of
predictions as a scatter plot and
also adding a line representing
perfect predictions (where predicted
equals actual) for reference. Labels,
a title, and a legend will be added
for clarity.
```

Generate a scatter plot of the actual test values (`y_test`) against the predictions from the Random Forest, KNN, and LSTM models. This will allow for a visual comparison of how well each model fits the data. Include appropriate labels, a title, and a legend to distinguish between the actual values and the predictions of each model.

Reasoning:

I will generate a scatter plot using `matplotlib.pyplot` to visualize the actual `y_test` values against the predictions from the Random Forest (`y_pred_rf`), KNN (`y_pred_knn`), and LSTM (`y_pred_lstm`) models. This will involve plotting each set of predictions as a scatter plot and also adding a line representing perfect predictions (where predicted equals actual) for reference. Labels, a title, and a legend will be added for clarity.

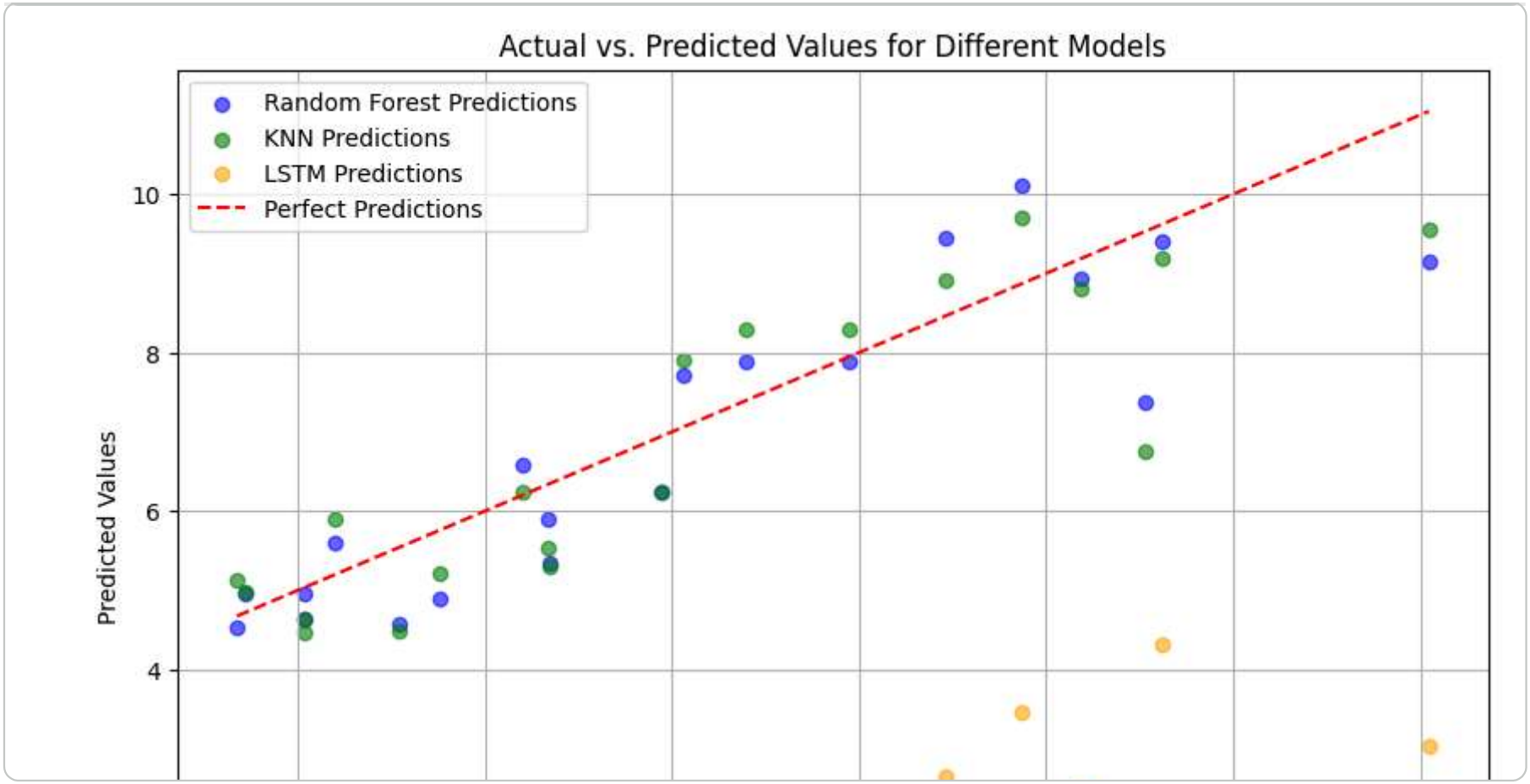
Plot Model Predictions

Subtask:

Generate a scatter plot of the actual test values (`y_test`) against the predictions from the Random Forest, KNN, and LSTM models. This will allow for a visual comparison of how well each model fits the data. Include appropriate labels, a title, and a legend to distinguish between the actual values and the predictions of each model.

Reasoning: I need to generate a code block that creates a scatter plot comparing the actual test values against the predictions from Random Forest, KNN, and LSTM models, along with a 'perfect predictions' line, and include all necessary plot elements like labels, title, and legend.

```
import matplotlib.pyplot as plt

# Ensure y_test is a 1D array for plotting consistently
y_test_flat = y_test.flatten()

plt.figure(figsize=(10, 7))

# Plot Random Forest Predictions
plt.scatter(y_test_flat, y_pred_rf.flatten(), alpha=0.6, label='Random Forest Predictions', color='blue')

# Plot KNN Predictions
plt.scatter(y_test_flat, y_pred_knn.flatten(), alpha=0.6, label='KNN Predictions', color='green')

# Plot LSTM Predictions (ensure y_pred_lstm is flattened and handled appropriately)
plt.scatter(y_test_flat, y_pred_lstm.flatten(), alpha=0.6, label='LSTM Predictions', color='orange')

# Plot Perfect Predictions line
plt.plot([y_test_flat.min(), y_test_flat.max()], [y_test_flat.min(), y_test_flat.max()],
         '--', color='red', label='Perfect Predictions')

plt.title('Actual vs. Predicted Values for Different Models')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.legend()
plt.grid(True)
plt.show()
```



Final Task

Subtask:

Review and present the generated plot, highlighting the visual performance comparison of the Random Forest, KNN, and LSTM models.

Close

```
## Summary:
```

```
### Data Analysis Key Findings

*   A scatter plot was successfully
generated, visually comparing the
actual test values (\`y_test\`)
against the predicted values from the
Random Forest, KNN, and LSTM models.
*   The plot included a "Perfect
Predictions" reference line, which
helps in visually assessing how
closely each model's predictions
align with the actual values.
*   The visualization facilitated a
direct comparison of the predictive
performance of the three models,
indicating how well each model fits
the data.

### Insights or Next Steps

*   The generated plot provides a
qualitative visual assessment of each
model's performance, allowing for
immediate identification of potential
biases (e.g., over-prediction,
```

Summary:

Data Analysis Key Findings

- A scatter plot was successfully generated, visually comparing the actual test values (`y_test`) against the predicted values from the Random Forest, KNN, and LSTM models.
- The plot included a "Perfect Predictions" reference line, which helps in visually assessing how closely each model's predictions align with the actual values.
- The visualization facilitated a direct comparison of the predictive performance of the three models, indicating how well each model fits the data.

Insights or Next Steps

- The generated plot provides a qualitative visual assessment of each model's performance, allowing for immediate identification of potential biases (e.g.